MACHINE LEARNING UPGRADE

데이터 분석 업그레이드
캐글에서 배우는 실전 기법

평가지표 · 특징 엔지니어링 · 검증 · 튜닝 · 앙상블

v02 · 2026.10

ⓒ 2026. DJ all rights reserved.

목 차

01 **평가지표 더 알아보기**
다중 클래스·다중 레이블 · F1 · MAP@K · 로그 손실

02 **데이터의 종류와 결측값**
정형·외부·시계열 데이터 · 결측값 처리

03 **수치형 특징 변환**
스케일링 · 로그 변환 · 클리핑 · 구간 분할 · 순위

04 **범주형 특징 변환**
원-핫 · 레이블 · 해싱 · 빈도 · 타깃 · 임베딩

05 **특징 조합과 텍스트 특징**
특징끼리 조합하기 · BoW · n-gram · tf-idf

06 **검증 방법과 리더보드**
교차 검증 · Trust your CV · 검증 과적합

07 **튜닝과 앙상블**
하이퍼파라미터 탐색 · 가중 평균 · 스태킹

\+ **정리**
한 장으로 보는 데이터 분석 업그레이드

ⓒ 2026. DJ all rights reserved.

00 학습 목표

머신러닝 기본 다음 단계: 대회와 실무에서 점수를 올리는 기법

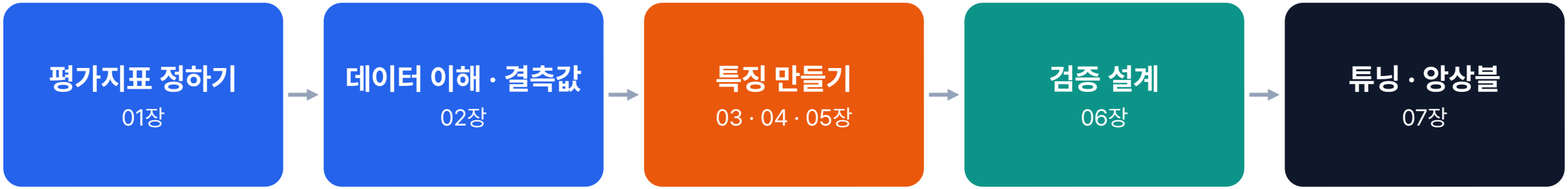
이 자료의 목표

- 머신러닝 기본에서 다루지 못한 추가 내용을 소개한다
- 이후 학습과 캐글 대회 참여에 도움이 되는 기법을 익힌다

참고 서적

- 『데이터가 뛰어노는 AI 놀이터, 캐글』
- 예제와 기법 이름은 이 책의 구성을 따른다

분석 흐름으로 보는 이 자료의 구성



특징 만들기(특징 엔지니어링)가 이 자료의 중심이다. 좋은 특징이 모델보다 점수를 더 크게 바꾸는 경우가 많다.

© 2026. DJ all rights reserved.

01

평가지표 더 알아보기

다중 클래스·다중 레이블 분류, F1의 확장, MAP@K, 로그 손실

01 평가지표 더 알아보기

분류 문제의 종류에 따라 평가지표가 달라진다

다중 클래스 분류 (multi-class)

- 샘플 하나가 여러 클래스 중 정확히 하나에 속한다
- 예: 사진 한 장 → 고양이 / 개 / 새 중 하나

다중 레이블 분류 (multi-label)

- 샘플 하나가 여러 레이블을 동시에 가질 수 있다
- 예: 영화 한 편 → 액션 + 코미디

문제 유형	대표 평가지표
이진 분류	로그 손실 (logloss)
다중 클래스 분류	다중 클래스 로그 손실 (multi-class logloss)
다중 레이블 분류	mean-F1, macro-F1
추천 — 순위를 매겨 제출	MAP@K
추천 — 순위 없이 제출	mean-F1, macro-F1, micro-F1

ⓒ 2026. DJ all rights reserved.

01 평가지표 더 알아보기

F1 점수를 여러 클래스로 확장하기 · 추천 문제의 MAP@K

mean-F1

- 샘플(행)마다 F1을 구한 뒤 평균
- 다중 레이블에서 사용

```
f1_score(y, p, average="samples")
```

macro-F1

- 클래스마다 F1을 구한 뒤 평균
- 모든 클래스를 똑같이 중요하게 본다

```
f1_score(y, p, average="macro")
```

micro-F1

- 모든 클래스의 TP·FP·FN을 합쳐 한 번에 F1 계산
- 샘플이 많은 클래스의 영향이 크다

```
f1_score(y, p, average="micro")
```

MAP@K (mean Average Precision at K)

- 추천 목록 상위 K개에 대해 정밀도의 평균(Average Precision)을 구하고, 다시 모든 샘플에 대해 평균한다
- 정답을 앞 순위에 둘수록 점수가 높다 — 순위를 매겨 제출하는 추천 문제의 대표 지표

ⓒ 2026. DJ all rights reserved.

01 평가지표 더 알아보기

이진 분류의 대표 지표: 로그 손실 (logloss)

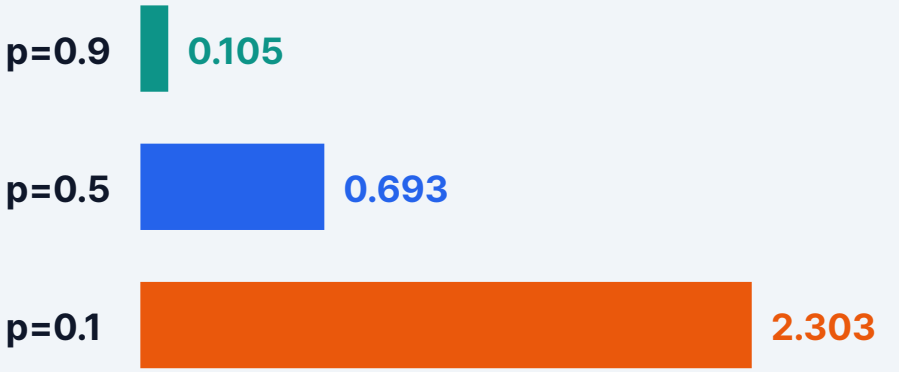
$$\text{logloss} = -\frac{1}{N}\sum\left[\, y_i \cdot \log(p_i) + (1 - y_i) \cdot \log(1 - p_i) \,\right]$$

읽는 법

- y_i: 실제 값 (0 또는 1), p_i: 1일 확률로 예측한 값
- 교차 엔트로피(cross-entropy)라고도 부른다
- 낮을수록 좋은 지표다
- 정답을 확신하며 맞히면 벌점이 작고, 확신하며 틀리면 벌점이 매우 크다

정답일 확률을 이렇게 예측했다면

샘플 하나의 로그 손실 = −log(p)




```
log_loss([1, 0], [0.9, 0.1])  # 0.105
```

ⓒ 2026. DJ all rights reserved.

02

데이터의 종류와 결측값

정형·외부·시계열 데이터, 그리고 비어 있는 값 다루기

02 데이터의 종류와 결측값

캐글에서 자주 만나는 세 가지 데이터

정형 데이터 (tabular)

- 행과 열로 된 표 형식 데이터
- 캐글에서는 보통 CSV 파일로 주어진다

```
id,age,city,target
1,35,서울,1
2,52,부산,0
```

외부 데이터

- 대회에서 준 데이터 밖의 데이터 (공개 데이터 등)
- 쓰기 전에 대회 규칙에서 허용하는지 반드시 확인한다

시계열 데이터

- 시간의 흐름에 따라 관측된 데이터
- 예: 일별 매출, 시간별 기온
- 검증할 때 시간 순서를 지켜야 한다 (06장)

```
date,store,sales
2026-10-01,A,120
2026-10-02,A,135
```

ⓒ 2026. DJ all rights reserved.

02 데이터의 종류와 결측값

결측값을 다루는 다섯 가지 방법

1 그대로 두기

GBDT(LightGBM, XGBoost 등)는 결측값을 그대로 학습한다. scikit-learn 1.4부터는 랜덤 포레스트도 지원한다

2 특정 값 넣기

-9999처럼 실제로 나오지 않는 값으로 채운다 (그대로 두는 것과 비슷한 효과)

3 대푯값으로 채우기

수치형은 평균이나 중앙값, 범주형은 가장 많이 나온 값(최빈값)

4 새 범주로 보기

범주형에서는 결측 자체를 '없음' 같은 새 범줏값으로 바꾼다

5 예측해서 채우기

결측이 있는 변수와 관련이 높은 다른 변수로 모델을 만들어 원래 값을 예측한다

```
df["age"].fillna(
    df["age"].median())

SimpleImputer(
    strategy="most_frequent")
```

ⓒ 2026. DJ all rights reserved.

02 데이터의 종류와 결측값

결측값 자체로 새 특징 만들기

나이	소득	직업	결측 개수	소득_결측
35	4200	회사원	0	0
—	3100	—	2	0
52	—	자영업	1	1

— 는 결측값. 오른쪽 두 열이 새로 만든 특징

```
df["결측 개수"] = df.isnull().sum(axis=1)
df["소득_결측"] = df["소득"].isnull().astype(int)
```

행마다 결측 개수 세기

- 결측값이 있는 특징의 수를 행별로 센다

결측 패턴으로 분류하기

- 여러 변수의 결측 조합이 몇 가지 패턴으로 나뉘면, 어느 패턴인지를 특징으로

결측 여부 0/1 특징

- 결측이면 1, 아니면 0인 특징을 만든다 (SimpleImputer(add_indicator=True))

ⓒ 2026. DJ all rights reserved.

03

수치형 특징 변환

스케일링 · 로그와 거듭제곱 변환 · 클리핑 · 구간 분할 · 순위 변환

03 수치형 특징 변환

그대로 써도 되지만, 알맞게 변환하면 더 좋은 특징이 된다

A. 표준화

평균 0, 분산 1로 바꾼다

```
StandardScaler()
```

B. 정규화 (최소-최대 스케일링)

최솟값 0, 최댓값 1 사이로 바꾼다

```
MinMaxScaler()
```

C. 로그 변환

log(x+1) 변환, 음수가 있으면 절댓값 로그 변환

```
np.log1p(x)
```

D. Box-Cox · Yeo-Johnson 변환

Box-Cox는 양수만, Yeo-Johnson은 0과 음수도 가능

```
PowerTransformer(method="yeo-johnson")
```

로그 변환 예: 값의 차이가 아주 큰 변수를 고르게 만든다

1	10	100	1000	→ log(x+1) →	0.69	2.40	4.62	6.91

© 2026. DJ all rights reserved.

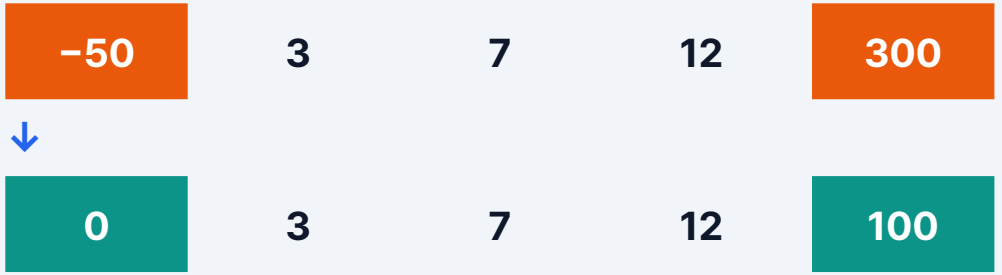
03 수치형 특징 변환

클리핑과 구간 분할

클리핑 (clipping)

- 상한과 하한을 정하고, 범위를 벗어난 값은 상한·하한 값으로 바꾼다
- 이상치의 영향을 줄일 때 쓴다

예: clip(0, 100)




```
s.clip(lower, upper)   # pandas
np.clip(x, lo, hi)     # numpy
```

구간 분할 (binning)

- 수치형 변수를 구간으로 나눠 범주형 변수로 바꾼다
- A. 같은 간격으로 나누기
- B. 분위수로 나누기 (구간마다 개수가 비슷)
- C. 구간 경계를 직접 정해서 나누기

```
pd.cut(x, 4)                    # A
pd.qcut(x, 4)                   # B
pd.cut(x, bins=[0, 19, 64, 120])  # C
np.digitize(x, bins)            # C
```

ⓒ 2026. DJ all rights reserved.

03 수치형 특징 변환

순위로 바꾸기와 RankGauss

순위 변환

- 수치형 변수를 크기 순서(순위)로 바꾼다
- 값의 크기 차이는 버리고 순서만 남긴다

10	200	30	5000
↓ 순위			
1	3	2	4

```
s.rank()
```

RankGauss

- 먼저 순위로 바꾼 뒤, 순서는 유지하면서 정규분포 모양이 되도록 강제로 변환한다
- 치우친 분포를 종 모양으로 만든다

```
QuantileTransformer(
    output_distribution="normal")   # 비슷한 변환
```

ⓒ 2026. DJ all rights reserved.

04

범주형 특징 변환

원-핫 · 레이블 · 특징 해싱 · 빈도 · 타깃 인코딩 · 임베딩

04 범주형 특징 변환

같은 데이터를 네 가지 방법으로 인코딩해 보기

원본		원-핫 인코딩			레이블	빈도	타깃
도시	y	대구	부산	서울	레이블	빈도	타깃
서울	1	0	0	1	2	3	0.67
부산	0	0	1	0	1	1	0.00
서울	1	0	0	1	2	3	0.67
대구	0	1	0	0	0	1	0.00
서울	0	0	0	1	2	3	0.67

원-핫 범주 수만큼 0/1 열이 생긴다

레이블 사전 순 번호 (대구 0, 부산 1, 서울 2)

빈도 등장 횟수 (서울 3번)

타깃 범주별 y의 평균 (서울 2/3 ≈ 0.67)

주의 타깃 인코딩을 이렇게 같은 데이터로 바로 계산하면 정답(y)이 특징에 새어 들어간다(누수). 실제로는 fold를 나눠 다른 fold의 y로 계산한다.

© 2026. DJ all rights reserved.

04 범주형 특징 변환

원-핫 인코딩과 레이블 인코딩

원-핫 인코딩 (One-Hot Encoding)

- 범주형 변수의 가장 대표적인 처리 방법
- 각 레벨마다 '그 레벨인지'를 0과 1로 나타내는 열을 만든다
- N개 레벨 → 0/1 열 N개. 이 열들을 가변수(dummy variable)라 한다
- 단점: 레벨이 많으면 특징 수가 크게 늘어난다

```
pd.get_dummies(df, columns=["도시"])
OneHotEncoder(handle_unknown="ignore")
```

레이블 인코딩 (Label Encoding)

- 각 레벨을 정수로 바꾼다 (레벨 5개 → 0~4)
- 보통 문자열을 사전 순으로 나열한 번호를 쓴다
- 숫자 크기에 의미가 없으므로 주로 트리 계열(GBDT 등) 모델에 쓴다

```
OrdinalEncoder()   # 특징(X)용
LabelEncoder()     # 정답(y)용
```

ⓒ 2026. DJ all rights reserved.

04 범주형 특징 변환

특징 해싱과 빈도 인코딩

특징 해싱 (Feature Hashing)

- 원-핫 인코딩은 열 수가 레벨 수와 같아진다. 특징 해싱은 이 수를 줄인다
- 변환 후 특징 수를 먼저 정하고, 해시 함수로 레벨마다 표시할 위치를 정한다
- 범주가 아주 많아 원-핫 열이 지나치게 많을 때 쓴다
- 다만 대회에서는 GBDT로도 어느 정도 대응되어 널리 쓰이지는 않는다

```
FeatureHasher(n_features=16, input_type="string")
```

빈도 인코딩 (Frequency Encoding)

- 각 레벨을 등장 횟수나 등장 비율로 바꾼다
- 레벨의 등장 빈도가 목적변수와 관련이 있을 때 효과적이다

```
df["도시"].map(df["도시"].value_counts())
```

ⓒ 2026. DJ all rights reserved.

04 범주형 특징 변환

타깃 인코딩과 임베딩

타깃 인코딩 (Target Encoding)

- 목적변수(y)를 이용해 범주형 변수를 수치로 바꾼다 (범주별 y 평균 등)
- 그냥 계산하면 정답이 새어 들어가 검증 점수가 부풀려진다
- 학습 데이터를 fold로 나누고, 다른 fold의 y로 값을 계산한다

```
TargetEncoder()   # scikit-learn 1.3+
# fit_transform이 fold 나누기를 자동 처리
```

임베딩 (Embedding)

- 단어나 범주처럼 띄엄띄엄한 값을 실수 벡터로 바꾼다
- 분산 표현(distributed representation)이라고도 한다
- 미리 학습된 단어 임베딩: word2vec, GloVe, fastText
- 요즘은 BERT 같은 트랜스포머 모델의 문장 임베딩도 많이 쓴다

ⓒ 2026. DJ all rights reserved.

04 범주형 특징 변환

테스트 데이터에만 있는 범주가 나오면

1

영향이 적으면 그대로

그런 행이 적으면 점수에 거의 영향을 주지 않는다. 먼저 영향이 큰지 확인한다

2

최빈값이나 예측으로 보완

가장 많이 나온 범주로 바꾸거나, 다른 변수로 범주를 예측해 채운다

3

평균에 가까운 값 넣기

타깃·빈도 인코딩처럼 수치로 바꿀 때는 그 변환의 평균에 가까운 값을 넣는다

```
enc = OneHotEncoder(handle_unknown="ignore")
# 처음 보는 범주는 모든 열이 0이 되어 오류 없이 변환된다
```

ⓒ 2026. DJ all rights reserved.

05

특징 조합과 텍스트 특징

변수끼리 조합하기 · 글을 숫자로 바꾸기

05 특징 조합과 텍스트 특징

여러 변수를 조합해 변수 사이의 상호작용을 표현한다

원칙 기계적으로 모두 조합하면 의미 없는 변수가 대량으로 생긴다. 데이터의 배경 지식으로 의미 있는 조합을 찾는다.

수치 × 범주

범주별로 수치 변수의 평균·분산 같은 통계량을 새 특징으로

```
df.groupby("도시")["가격"]
  .transform("mean")
```

수치 × 수치

사칙연산으로 새 특징. 예: Zillow Prize 대회 — 집 면적 ÷ 방 개수

```
df["면적"] / df["방수"]
```

범주 × 범주

여러 범주형 변수를 합쳐 새 범주형 변수를 만든다

```
df["도시"] + "_" + df["상품"]
```

행의 통계량

행마다 여러 변수의 결측·0·음수 개수, 평균·분산·최대·최소 (일부 변수로 좁혀서)

```
df[cols].mean(axis=1)
```

ⓒ 2026. DJ all rights reserved.

05 특징 조합과 텍스트 특징

자연어 처리 기법: 글을 숫자로 바꾸기

Bag-of-Words 예: 문장마다 단어 등장 횟수를 센다

문장 1 나는 사과를 좋아한다

문장 2 사과를 사고 사과를 먹었다

	나는	먹었다	사고	사과를	좋아한다
문장 1	1	0	0	1	1
문장 2	0	1	1	2	0

```
CountVectorizer()        # BoW
TfidfVectorizer()        # tf-idf
```

Bag-of-Words (BoW)

- 단어로 나누고 순서와 상관없이 등장 횟수를 센다

n-gram

- 단어 하나가 아니라 이어지는 단어 묶음 단위로 센다 (예: '사과를 사고')

tf-idf

- BoW 행렬을 변환해, 여러 문서에 흔한 단어의 가중치는 낮춘다

단어 임베딩

- 단어를 의미를 담은 수치 벡터로 바꾼다 (04장 임베딩)

ⓒ 2026. DJ all rights reserved.

06

검증 방법과 리더보드

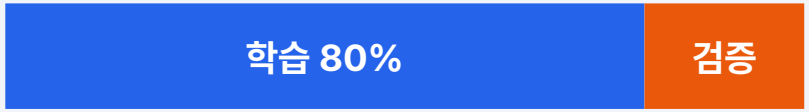
교차 검증 · Trust your CV · 검증 데이터 과적합

06 검증 방법과 리더보드

홀드아웃 검증과 K-겹 교차 검증

홀드아웃 검증

- 가장 간단한 방법
- 학습 데이터의 일부를 학습에 쓰지 않고 검증용으로 남겨 둔다



한 번만 나누므로 빠르지만, 어떻게 나눴느냐에 따라 점수가 흔들릴 수 있다

K-겹 교차 검증 (K-Fold)

- 데이터를 K개로 나누고, 검증 fold를 바꿔 가며 K번 반복한다



최종 점수 = 5번 검증 점수의 평균

ⓒ 2026. DJ all rights reserved.

06 검증 방법과 리더보드

상황에 맞는 검증 방법 고르기

방법	언제 쓰나	scikit-learn
K-겹	기본 교차 검증	`KFold`
층화 K-겹	분류에서 각 fold의 클래스 비율을 맞출 때	`StratifiedKFold`
그룹 K-겹	같은 그룹(예: 같은 고객의 여러 행)이 학습·검증에 나뉘면 안 될 때	`GroupKFold`
LOO	데이터가 아주 적을 때 하나씩 빼서 검증 (계산량이 크다)	`LeaveOneOut`
시계열 분할	과거로 학습하고 미래로 검증해야 할 때	`TimeSeriesSplit`

시계열 분할은 v02에서 추가 (02장 시계열 데이터와 연결)

ⓒ 2026. DJ all rights reserved.

06 검증 방법과 리더보드

리더보드 점수보다 교차 검증을 믿어라 — Trust your CV

캐글의 'Trust Your CV'

- Public 리더보드 점수에 흔들리지 말고, 검증으로 모델을 평가해 일반화 성능이 좋은 모델을 찾는 것이 중요하다

검증 점수와 Public 리더보드 점수가 잘 맞지 않을 때 생각해 볼 원인

1. **우연에 의한 점수의 편차**
2. **검증 데이터와 테스트 데이터의 분포가 다르다**
3. **검증 설계가 부적절해 일반화 성능을 제대로 평가하지 못한다**

잘 맞는다면 학습·테스트 데이터의 성질이 가깝고 검증도 잘 된 것이다. 안심하고 그 검증을 계속 쓴다.

ⓒ 2026. DJ all rights reserved.

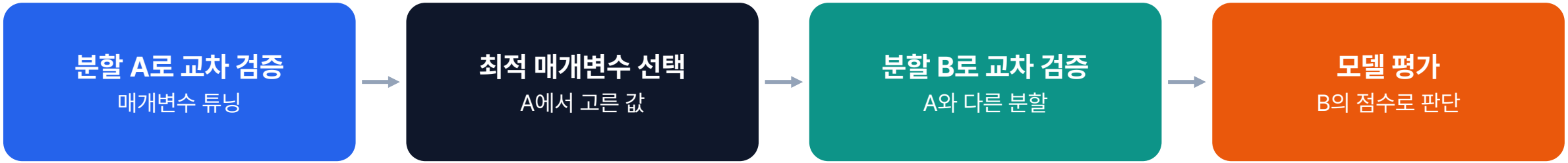
06 검증 방법과 리더보드

검증 데이터와 Public 리더보드에 과적합하지 않기

지나친 시도에 따른 과적합

- 매개변수를 수없이 바꿔 보며 검증 점수만 보고 고르면, 검증 데이터에 과적합할 수 있다
- 제출할 때마다 검증 점수와 Public 리더보드 점수를 그래프로 그려 어긋나는지 살핀다

교차 검증의 분할을 바꿔 보기



튜닝에 쓴 분할과 평가에 쓰는 분할을 다르게 하면, 튜닝 과정에서 생긴 과적합을 걸러낼 수 있다

ⓒ 2026. DJ all rights reserved.

07

튜닝과 앙상블

하이퍼파라미터 탐색 · 튜닝 요령 · 가중 평균 · 스태킹

07 튜닝과 앙상블

하이퍼파라미터를 찾는 세 가지 방법

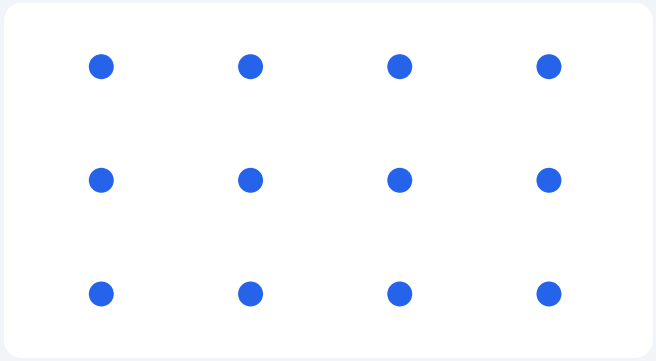
수동 조정

경험으로 값을 바꿔 가며 확인



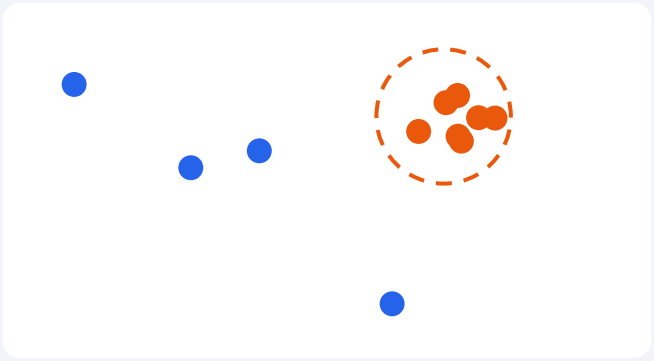
그리드 · 랜덤 서치

정해 둔 격자 전체 또는 무작위 조합을 시험



베이즈 최적화

앞선 결과를 보고 좋은 곳 근처를 집중 탐색



베이즈 최적화 라이브러리

hyperopt가 많이 쓰였고, 2018년 공개된 Optuna도 현재 많이 쓰인다

```
def objective(trial):
    d = trial.suggest_int("max_depth", 2, 10)
    return my_cv_score(d)        # 직접 만든 검증 함수
study = optuna.create_study(direction="maximize")
study.optimize(objective, n_trials=50)
```

ⓒ 2026. DJ all rights reserved.

07 튜닝과 앙상블

매개변수 튜닝의 요령

1 **중요한 매개변수부터**

중요한 것과 덜 중요한 것을 나누고, 중요한 매개변수부터 조정한다

2 **범위 끝에 좋은 값이 몰리면**

탐색 범위의 상한이나 하한에 좋은 점수가 몰려 있으면 범위를 넓혀 다시 탐색한다

3 **난수 시드 고정**

학습할 때 난수 시드(random_state)를 정해 두어 결과를 다시 만들 수 있게 한다

4 **시드를 바꿔 점수 변화 보기**

모델과 fold 분할의 시드를 바꿨을 때의 점수 변화와 비교하면, 개선이 매개변수 덕분인지 우연인지 가늠할 수 있다

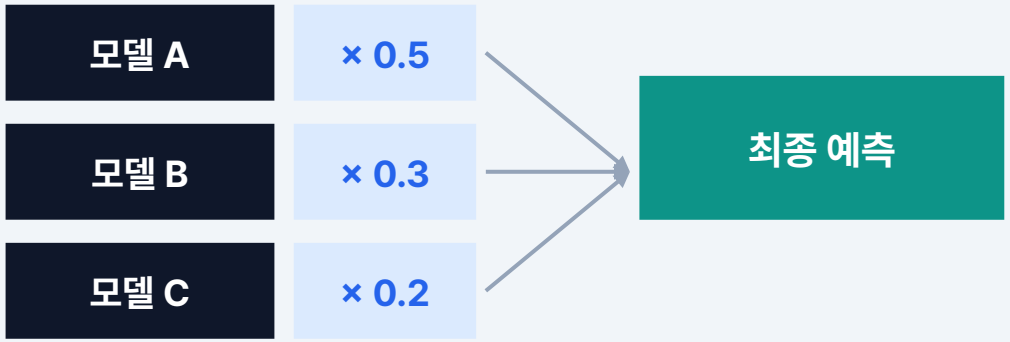
ⓒ 2026. DJ all rights reserved.

07 튜닝과 앙상블

간단한 앙상블: 평균과 가중 평균, 스태킹

평균과 가중 평균

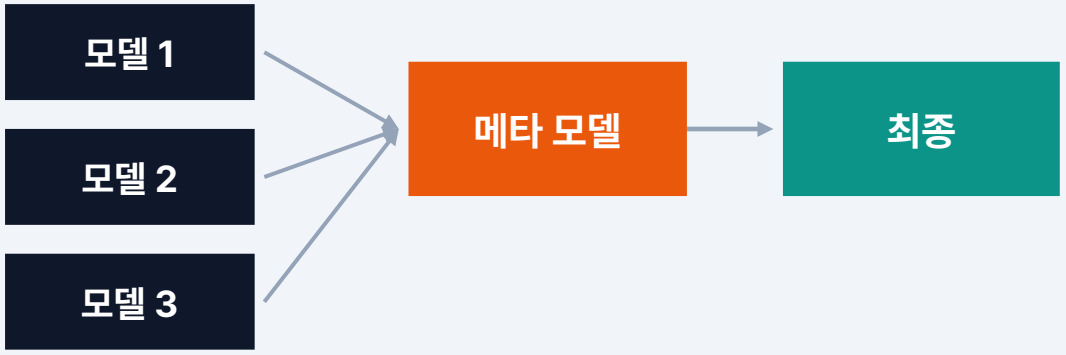
- 여러 모델의 예측을 평균한다. 가중치는 모델 성능을 보며 정한다
- 점수가 가장 높아지는 가중치를 scipy.optimize 등으로 찾을 수도 있다



주의 Public 리더보드만 보고 가중치를 정하면 리더보드에 과적합할 수 있다 (06장)

스태킹 (Stacking)

- 둘 이상의 모델 예측을 다시 입력으로 쓰는 메타 모델로 최종 예측을 만든다




```
StackingClassifier(estimators=[...],
    final_estimator=LogisticRegression())
```

ⓒ 2026. DJ all rights reserved.

07 튜닝과 앙상블

앙상블할 모델을 고를 때의 팁

1

검증 결과를 로그로 남기기

각 모델의 검증 점수를 기록해 한눈에 비교할 수 있게 한다

```
print(f"{name}: {score:.4f}")
```

2

모델의 다양성 보기

예측값끼리의 상관계수를 구하거나 산점도를 그린다. 서로 다른 모델일수록 앙상블 효과가 크다

```
np.corrcoef(pred_a, pred_b)
```

3

검증 점수와 리더보드 비교

모델마다 검증 점수와 단독 제출 시 Public 리더보드 점수를 그래프로 그려 본다

```
plt.scatter(cv_scores, lb_scores)
```

© 2026. DJ all rights reserved.

정리 한 장으로 보는 데이터 분석 업그레이드

평가지표

문제 유형에 맞는 지표를 먼저 정한다.

로그 손실 · macro/micro/mean-F1 · MAP@K

특징 엔지니어링

결측값 처리, 수치 변환(스케일링·로그·클리핑·구간·순위),

범주 인코딩(원-핫·레이블·빈도·타깃), 조합, 텍스트

검증

상황에 맞는 교차 검증을 고르고, Trust your CV.

튜닝용 분할과 평가용 분할을 나눠 과적합을 막는다.

튜닝과 앙상블

중요한 매개변수부터, 베이즈 최적화(Optuna 등)로 탐색.

다양한 모델을 가중 평균하거나 스태킹한다.